Módulo 2 — Grupo: JOULIE FARADAY

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

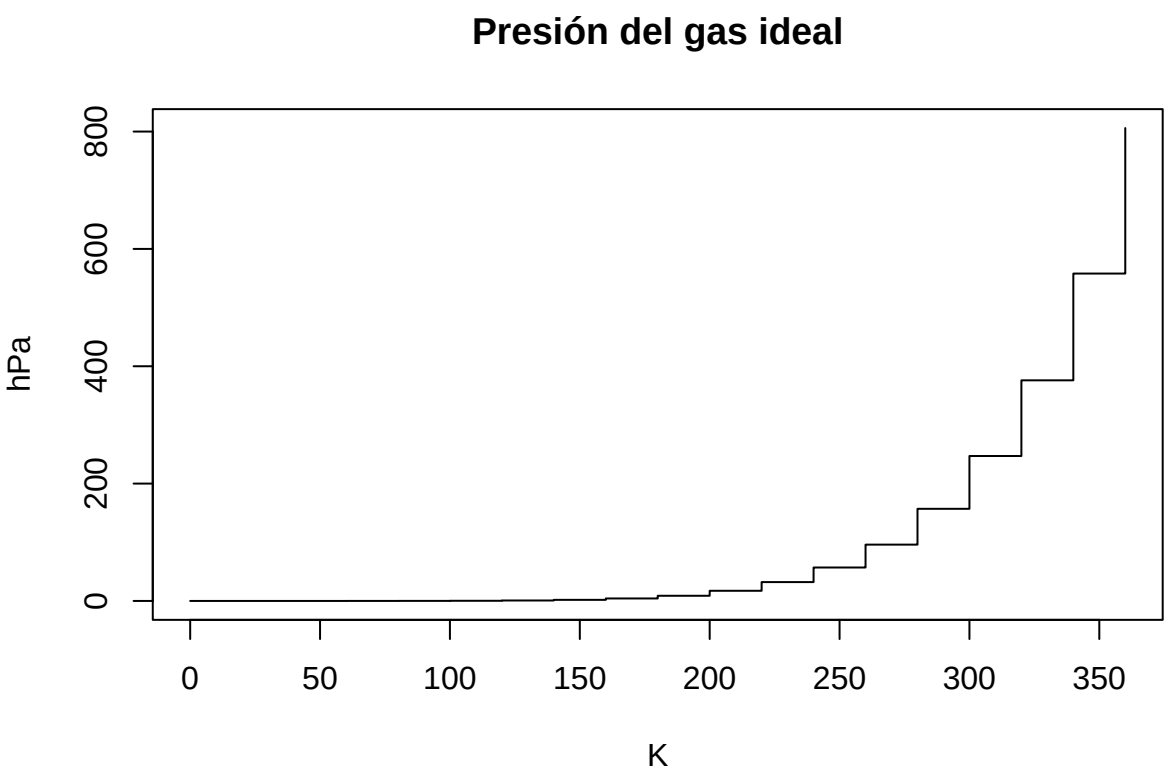
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 20.061368 31.072446 23.609466 28.571910 24.387037 21.122708 25.981135
##   [8] 28.673787 16.612053 30.397840 25.803805 18.210410 20.849177 20.505902
##  [15] 20.856296 23.489126 15.770792 16.799540 24.395372 20.053516 17.716645
##  [22] 22.561064 17.894832 11.313017 29.505211 25.211111 25.578325 28.743611
##  [29] 21.636658 24.419030 23.645768 16.288469 22.665683 23.495965 23.355102
##  [36] 18.567770 12.189736 15.942736 26.617410 21.612530 25.080583 18.412480
##  [43] 17.387964 20.334913 20.647824 21.076585 17.250819 17.452381 25.673222
##  [50] 18.631377 17.201720 18.809834 21.003219 23.623331 32.072060 22.396248
```

```
##  [57] 14.894817 25.823050 20.593182 14.876930 18.765065 26.954915 28.763166
##  [64] 20.818002 21.944416 19.494416 27.208758 14.391635 24.799157 19.386811
##  [71] 24.780927 22.471543 13.880580 20.374310 22.731779 20.154081 24.530432
##  [78] 17.287789 23.282567 21.112481 23.745447 11.626481 24.193824 25.418276
##  [85] 20.913902 26.581286 11.544197 24.318987 19.786137 17.418439 17.257969
##  [92] 28.143371 18.366343 21.855974 14.334965 32.151453 19.860716 13.996396
##  [99] 18.488622 17.796703 19.962270 23.239908 25.247169 15.777170 25.871386
## [106] 32.843849 21.737689 28.809128 18.580032 29.075685 26.304326 15.791156
## [113] 25.736858 26.207752 20.423705 28.994286 20.693476 29.922514 30.294650
## [120] 20.147187 14.150965 19.732172 21.253752 24.351961 24.272753 24.742801
## [127] 18.179797 15.333438 27.266083 18.345935 24.281881 27.381438 27.975638
## [134] 29.046145 29.981828 20.280719 17.308121 17.911146 16.358312 16.702556
## [141] 26.708589 22.708933 18.029526 17.288499 13.188981 21.019275 18.006382
## [148] 24.712645 22.913767 30.174056 18.698640 20.968595 22.736300 28.207044
## [155] 22.433957 22.357127 20.581002 20.474142 30.061861 29.535913 30.128172
## [162] 23.899066 21.546685 25.807473 20.126945 20.801752 17.193839 29.570292
## [169] 25.728632 16.148831 20.029547 18.486791 28.839198 22.392689 13.595412
## [176] 19.240779 27.020644 19.755918 18.926464 22.004194 29.054172 19.087455
## [183] 18.037352 23.167152 27.166411 25.073461 19.468040 28.013199 21.046704
## [190] 18.986633 19.374962 31.251523 13.443874 26.062531 20.298586 29.225828
## [197] 20.558324 23.646296 26.518040 22.867074 15.126243 20.200910 25.647042
## [204] 15.441135 26.563239 15.660188 19.324111 25.018674 22.056902 17.269030
## [211] 22.450423 32.706212 22.507263 22.027730 18.576667 22.926592 29.903764
## [218] 19.912044 22.847371 19.650710 26.723980 19.658997 28.154855 34.884639
## [225] 24.756561 24.203987 21.503832 26.971123 20.367992 29.833115 27.405317
## [232] 23.914961 23.478663 22.399036 24.274781 20.422089 17.957831 20.083047
## [239] 23.831184 11.566676 23.750000 19.081865 24.503888 22.945951 20.506570
## [246] 23.285033 21.857352 26.710375 28.368444 26.258608 27.071389 28.637652
## [253] 29.985750 18.935241 22.555056 31.214663 24.714333 29.855144 19.489453
## [260] 26.499915 29.350645 20.762821 22.170243 23.336447 21.056923 15.884774
## [267] 23.179455 21.817525 24.002503 29.400956 24.107021 20.779708 17.600972
## [274] 19.724630 27.177928 23.091929 20.758734 22.312704 27.736485 20.654843
## [281] 15.572198 24.005331 18.242774 17.059459 23.844082 22.395871 12.698961
## [288] 23.877099 22.092844 24.842195 22.778510  9.640319 14.017057 19.409857
## [295] 18.499660 27.249151 13.853201 21.239395 26.158894 26.109634 22.557458
## [302] 17.463948 26.191765 12.445833 22.171639 23.032153 21.601860 36.201867
## [309] 21.457809 19.275139 29.468286 16.859396 20.913322  8.547867 32.424329
## [316] 13.465249 26.422813 19.955100 28.168692 17.574710 20.418138 20.773232
## [323] 29.531321 28.142978 29.103885 19.773585 25.897067 24.914499 18.934812
## [330] 16.004335 13.276786 17.446570 33.161426 23.234346 24.295535 19.233855
## [337] 18.681585 22.739029 10.659896 14.202009 25.631063 22.485608 24.584361
## [344] 21.305672 26.702970 22.727435 13.201321 32.563322 22.018102 30.029743
```

```r
w1 <- length(z1)
w1
```

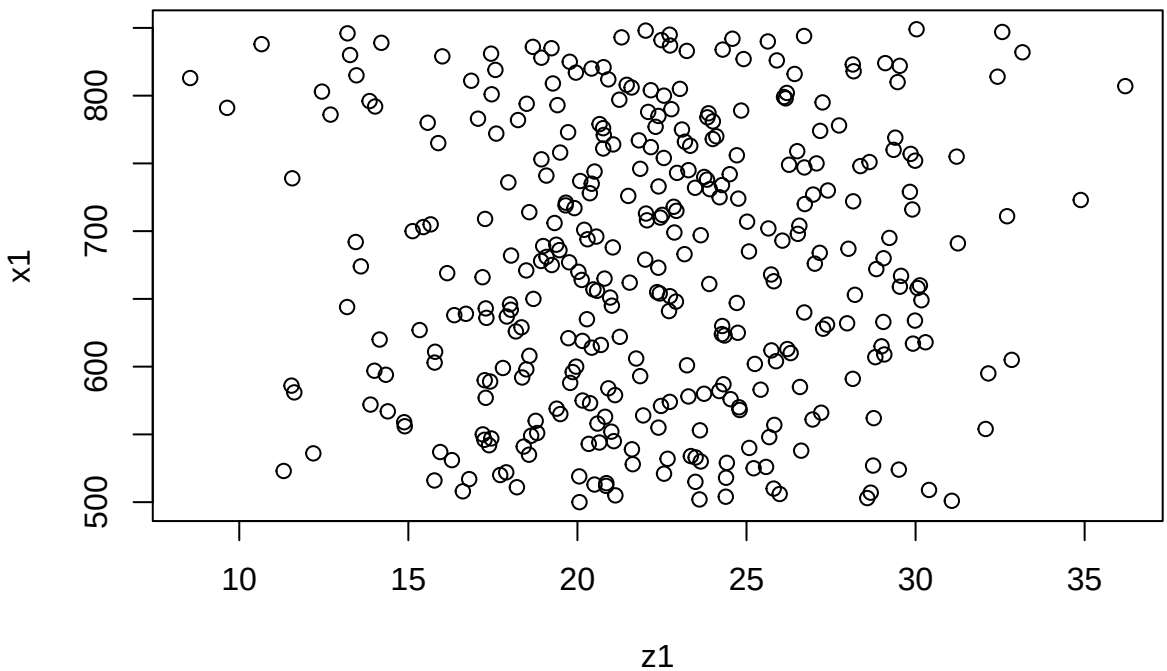
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

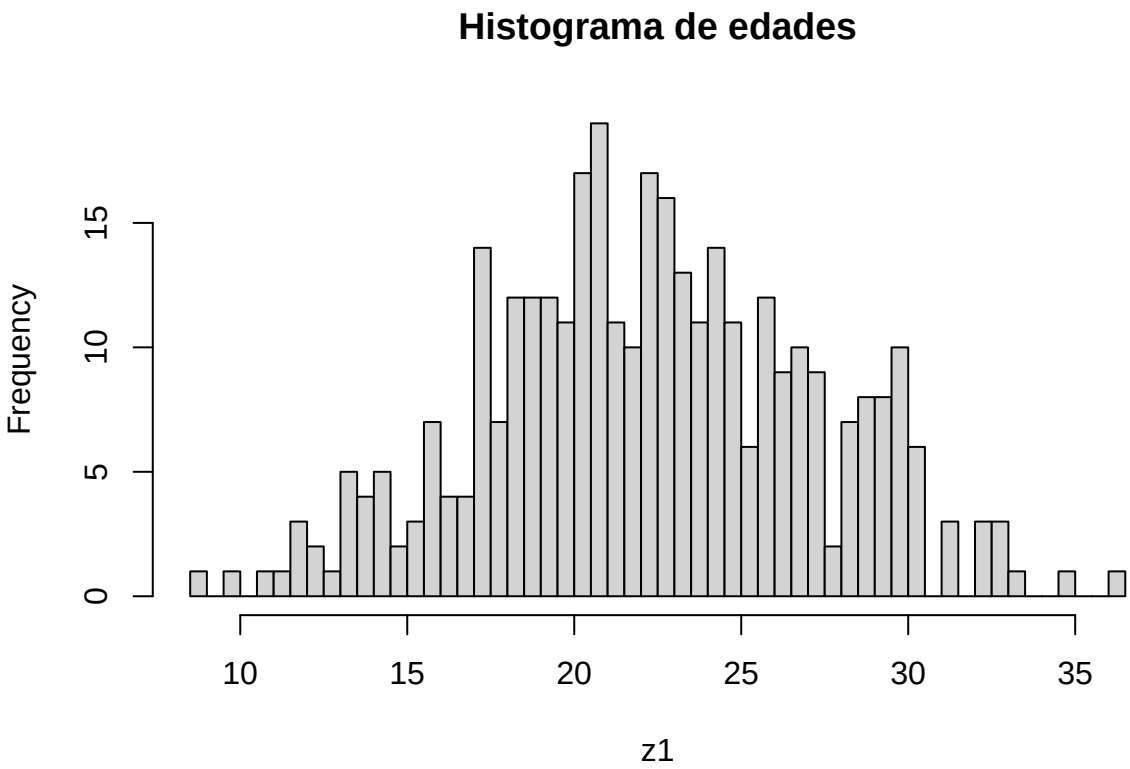
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



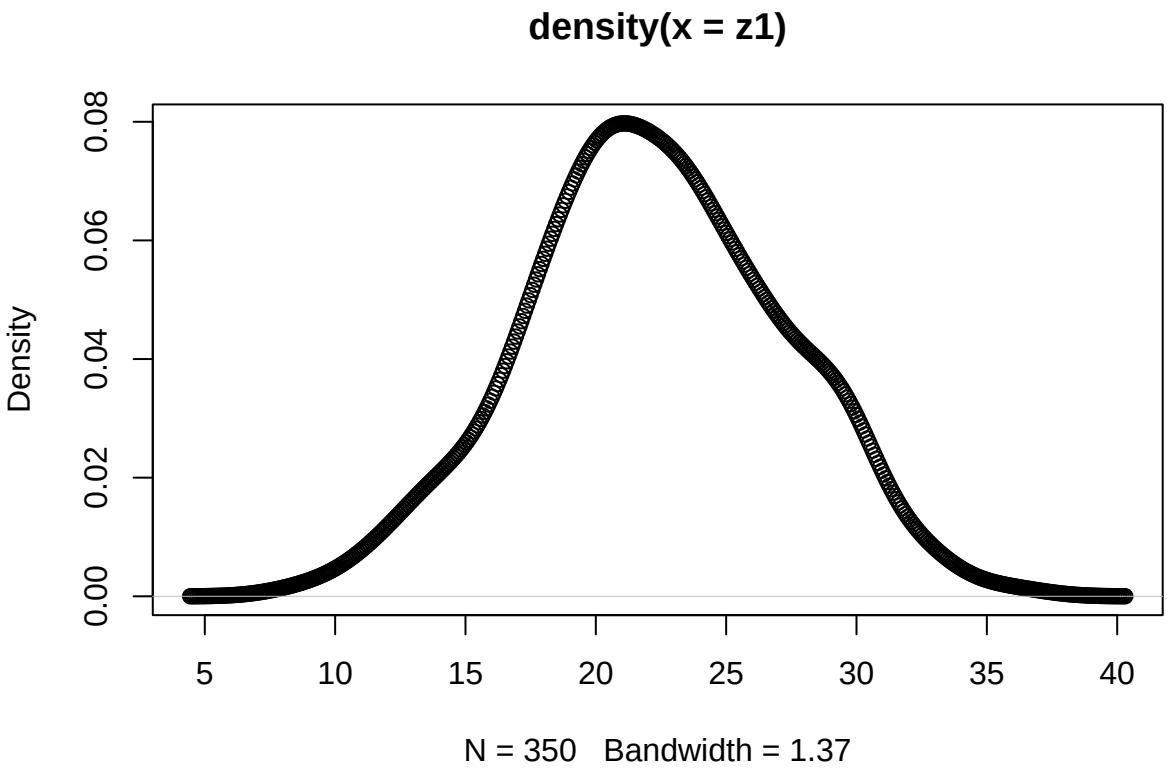
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.37
##
##        x                y
##  Min.   : 4.439   Min.   :9.641e-06
##  1st Qu.:13.407   1st Qu.:2.121e-03
##  Median :22.375   Median :1.802e-02
##  Mean   :22.375   Mean   :2.782e-02
##  3rd Qu.:31.343   3rd Qu.:5.052e-02
##  Max.   :40.311   Max.   :7.974e-02
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
secuencia_dni = list(range(1, dni + 1))
```

```python
# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.317372 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.
```

```r
#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.004 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.007588148 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.002122402 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
```

```
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
```

```r
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  22.616   0.068  22.950
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.017   0.000   0.017
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

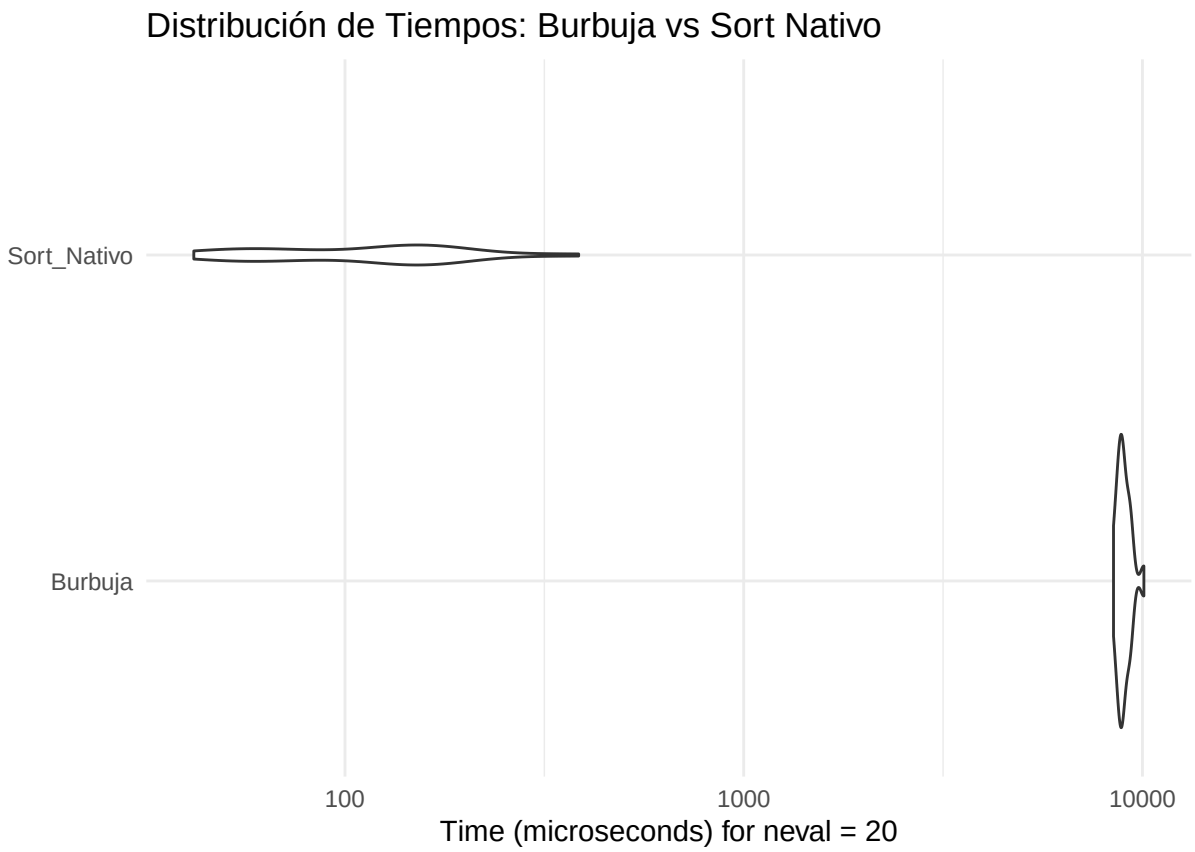
```
## [1] 500000500000
```

```r
suma2
```

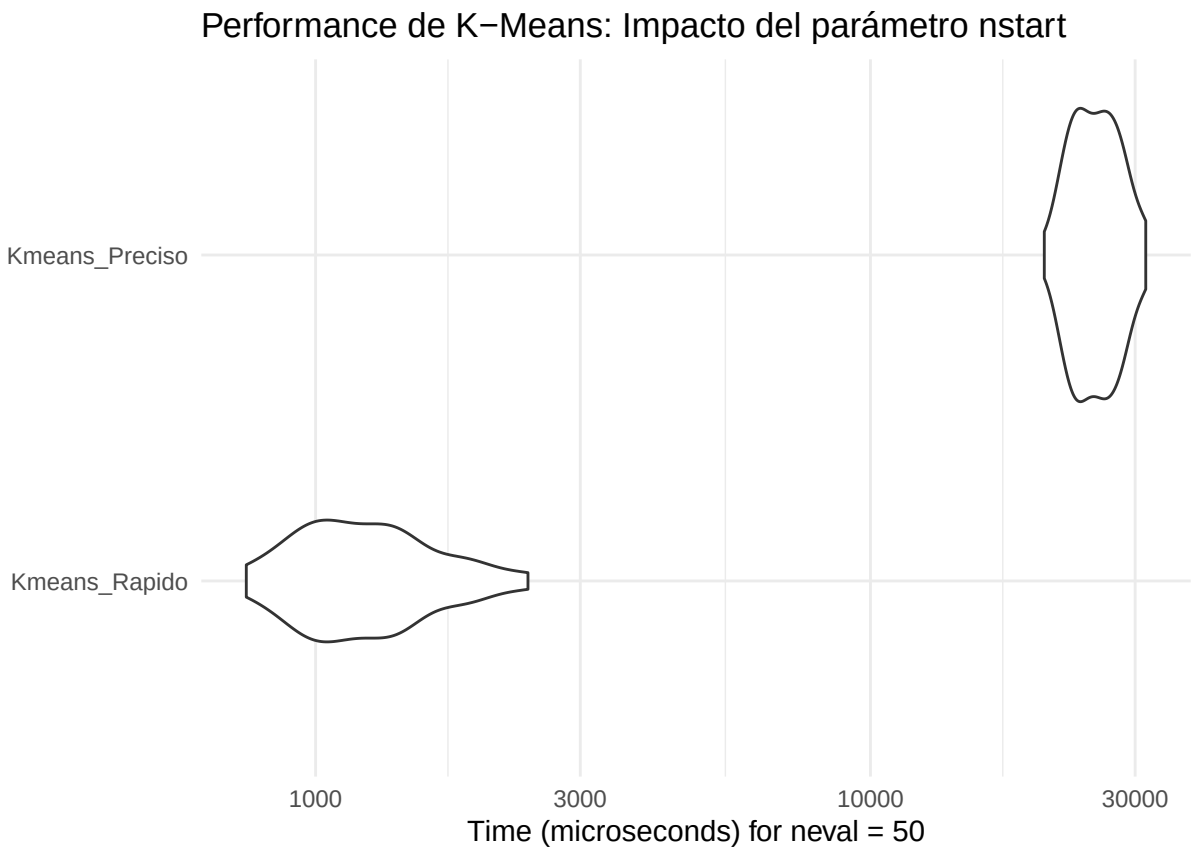
```
## [1] 500000500000
```

Análisis de Rendimiento: Ordenamiento y Agrupación

```r
library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(
  Burbuja = burbuja(muestra),
  Sort_Nativo = sort(muestra),
  times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()
```




```r
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()
```



Conclusión final

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos empíricamente que la calidad de un código no radica solo en llegar al resultado correcto, sino en cómo optimiza el tiempo y los recursos del sistema.

La incorporación de los gráficos de violín resultó fundamental para visualizar estadísticamente no solo la media de ejecución, sino la estabilidad y distribución temporal de procesos analíticos complejos, como la comparación de métodos de ordenamiento y la medición de performance del agrupamiento K-Means. Es fundamental destacar que los tiempos absolutos medidos y la forma exacta de la "panza" de los violines serán diferentes en cada computadora; estos resultados variarán dinámicamente según las propiedades físicas y la capacidad de procesamiento (hardware) del equipo donde se compile el documento.

Sin embargo, más allá de la máquina específica que se utilice, el principio de optimización estructural se mantiene inalterable. El ejercicio de "La penitencia de Newton" ilustró perfectamente este concepto: mientras la fuerza bruta de un bucle iterativo demanda un costo computacional inmenso para realizar operaciones masivas, la aplicación de un modelo matemático sólido resuelve el problema en fracciones de segundo.

En definitiva, la evaluación de estos algoritmos nos confirma que la herramienta de optimización más poderosa en la programación moderna no es la computadora que tengamos, sino el razonamiento lógico, físico y matemático que aplicamos antes de escribir la primera línea de código.